Model	N_{freq}	N_{emb}	N_{layer}	H	N_{qkv}	N_{ff}	N_{MLP}	seed
`Baseline`	32	512	10	12	64	512	2×512	100
`Alternative`	32	512	8	12	64	512	2×512	100
`Large`	32	1024	12	16	64	512	2×512	100

Table 6: Model hyperparameters for main experiments (i.e. other than training stability). `Baseline` is used for the main results presented in section 7 (short). `Alternate` and `Large` are presented in appendix C to show reproducibility of observations.

Model	N_{freq}	N_{emb}	N_{layer}	H	N_{qkv}	N_{ff}	N_{MLP}	seed
`All`	16	-	-	12	-	512	3×512	1,2

Table 7: Model hyperparameters for training stability experiments. Empty parameters are varied per-model and displayed in Figure 24.

apply this scale factor. This ensures that the model is initialised with a standard deviation of σ at that point.

We apply `Checkpoint` layers to the token embeddings x_{type} ($\sigma_{type} = 0.5$), and the initial embeddings x ($\sigma_x = 1.0$), ensuring they are relatively balanced and unit scale. In every attention layer, we apply `Checkpoint` layers to re-calibrate the possibly-Pre-normalised embeddings to σ_x immediately before applying the W_Q, W_K, and W_V operators. This counteracts the effect that transformer necessarily increases the embedding variance throughout the model at initialisation. We apply `Checkpoint` layers to w_t in every attention layer, with constant $\sigma_w = 0.1$. This controls the variance on the initial-state attention distribution. We apply `Checkpoint` layers to Δx in every attention layer, with constant $\sigma_{\Delta x} = 0.05$, calibrating it with respect to x.

In the attention layer, we use uniform initialisation of the weight matrices W_Q, W_K, W_V, and W_O. The limits are configured to ensure that the initial state standard deviations on w_t and Δx are close to their target values. Defining $\sigma_{qk} \triangleq \sqrt[4]{\frac{\sigma_w}{N_{qkv}^3}}$, the limits are calculated as follows:

Weight	Limits
W_Q	$\pm\sqrt{3}\sigma_{qk}$
W_K	$\pm\sqrt{3}\sigma_{qk}$
W_V	$\pm\sqrt{\frac{3}{N_{qkv}}}$
W_O	$\pm\sqrt{\frac{3}{HN_{qkv}}}$

However, we note that this initialisation is superseded by the calibration of the `Checkpoint` layers for determining the initial state, and we include it only to promote numerical stability. All other feed-forward layers use `Glorot uniform` [66] initialisation, as implemented in `Keras` [67]. Normalisation gain parameters are initialised to 1 and biases, where used, to 0.

A.7 Training algorithm

We train using the `AdamW` optimiser [68] with learning rate 3×10^{-4} and weight decay of 0.01, with all other parameters following their default values in `TensorFlow+Keras v2.15.0`. Each epoch consists of 2000 batches of 128 datapoints. For the main experiments and model variations, we use an *adaptive learning rate decay* strategy. This means that the learning rate is multiplied by a factor of 0.5 if the training loss does not improve for 3 consecutive epochs. We find that this balances training speed with improved performance by using small learning rates later in training. Training is halted after two days of wall time, which we observe to allow model convergence, as shown in Figure 8. For the model stability scan, training is run for 60 hours, and learning rate is not allowed to decay (stability with respect to learning rate being one of the targets of study).

A.8 Computational resources

The main experiments are all performed on a single `Nvidia v100-SXM2-16GB (Volta)` GPU. The scan of models used for the stability analysis were trained on a batch cluster with a variety of compute

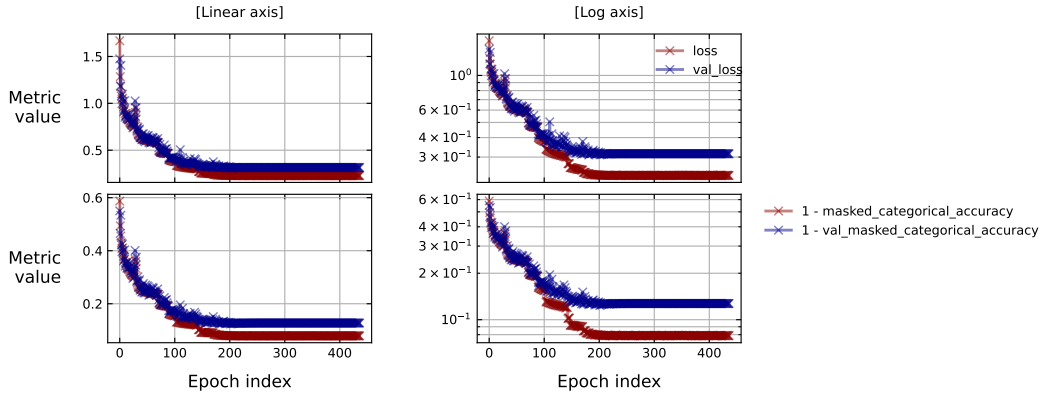

Figure 8: Model training curves for the `Baseline` `Pre-Norm` configuration.

nodes, using 8 cores per training run. A representative compute node is `2×12-core Intel Xeon E5-2690 v3 @ 2.60GHz + 128GB RAM`.

A.9 Environment details

The main contributing package versions are as follows:

Package	Version
Python	`3.11.5 (main, Sep 11 2023, 13:54:46) [GCC 11.2.0]`
TensorFlow	`2.15.0`
Keras	`2.15.0`
NumPy	`1.26.2`

B Extended main experiments

This appendix provides an extended explanation of the experimental results in section 7.

B.1 Embedding structure

Figure 1 presented the spread of embedding L_2-norms as a function of model depth. Let us now describe in detail how this plot was made. Figure 9 shows the distribution of embeddings at the input to every attention layer, for `Baseline` models trained using `Pre-Norm` (top) and `QKV-Norm` (bottom). Colours represent the initial token type corresponding to that embedding. Asterisks denote tokens in the *answer*, with all labels denoting the *question*.

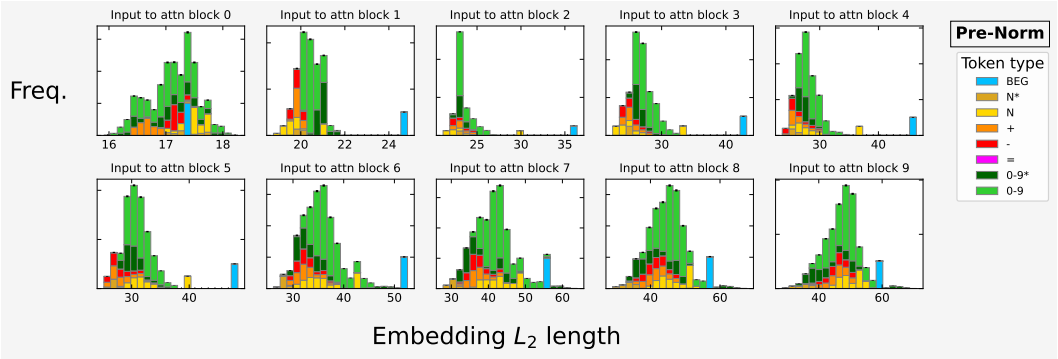

Figure 9: Distribution of embedding L_2-norms at different model depths using the `Baseline` `Pre-Norm` model.

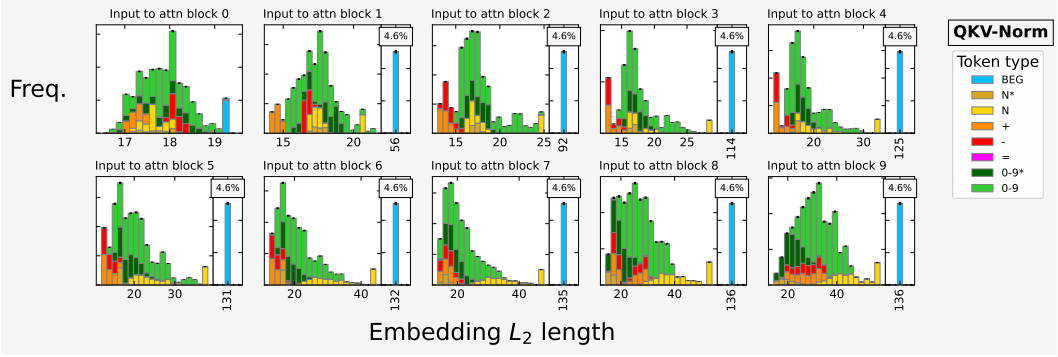

Figure 10: Distribution of embedding L_2-norms at different model depths using the `Baseline` `QKV-Norm` model.

We see that the begin-sequence token (`BEG`) is often separate from the distribution, which may be because it remains non-annotated and fulfils a qualitatively different role. We remove this from our estimates to avoid erroneously inflating the spread. Interestingly, `BEG` still tends to be close to the main bulk for `Pre-Norm`, but can be very far for `QKV-Norm`, and we have to use overflow panels to capture it. This is consistent with our hypothesis that `Pre-Norm` stability requires embeddings to have similar norms, whilst `QKV-Norm` does not require this.

It is not sufficient to simply measure the spread of Figures 9 and 10, because an attention head may not be sensitive to all embeddings in the layer. The easiest way to account for this is to weight every embedding according to its assigned attention. Secondly, the distribution is expected to be narrow only on a per-head basis, and there is no reason why distinct heads cannot be centred around different medians. We therefore calculate the weighted distribution of embeddings on a per-head basis, as shown in Figures 11-12.